Lab6

無人機手動控制 (20%)
無人機自動追蹤 (80%)

手動控制

```
def keyboard(self, key):
    #global is_flying
    print("key:", key)
    fb_speed = 40
    lf_speed = 40
    ud_speed = 50
    degree = 30
    if key == ord('1'):
        self.takeoff()
        #is_flying = True
    if key == ord('2'):
        self.land()
        #is_flying = False
    if key == ord('3'):
        self.send_rc_control(0, 0, 0, 0)
        print("stop!!!!")
    if key == ord('w'):
        self.send_rc_control(0, fb_speed, 0, 0)
        print("forward!!!!")
    if key == ord('s'):
        self.send_rc_control(0, (-1) * fb_speed, 0, 0)
        print("backward!!!!")
```

將 keyboard_djitellopy.py 中的 function import 到你的code上 (lab6.py)

之後在你的while迴圈下面加入此行，並確保其有最高的優先權，以便強制控制無人機

```
if key != -1:
    keyboard(drone, key)
```

Tello 控制 速度 function

```
send_rc_control(self, left_right_velocity,
forward_backward_velocity, up_down_velocity,
yaw_velocity)
```

Send RC control via four channels. Command is sent every self.TIME_BTW_RC_CONTROL_COMMANDS seconds.

Parameters:

Name	Type	Description	Default
`left_right_velocity`	`int`	-100~100 (left/right)	*required*
`forward_backward_velocity`	`int`	-100~100 (forward/backward)	*required*
`up_down_velocity`	`int`	-100~100 (up/down)	*required*
`yaw_velocity`	`int`	-100~100 (yaw)	*required*

務必確認輸入的值為整數 (int)

```
drone.send_rc_control(0, int(z_update) , int(y_update ) , int(yaw_update) )
```

- 所有參數範圍皆為 -100 ~ 100，且必須為整數(int)。
- 數值的絕對值越大，代表移動或旋轉速度越快，正負值代表著不同的方向，可看左圖括號內來選擇。
- 輸入 0 代表該方向不施加 速度 命令，因此可使用 send_rc_control(0, 0, 0, 0) 來停止所有方向的移動指令。

Tello 控制 移動距離 function

`move(self, direction, x)`

Tello fly up, down, left, right, forward or back with distance x cm. Users would normally call one of the move_x functions instead.

Parameters:

Name	Type	Description	Default
`direction`	`str`	up, down, left, right, forward or back	*required*
`x`	`int`	20-500	*required*

- 距離與角度參數皆須為整數(int)。
- 這裡的函式都會依照指定距離或角度執行動作，與持續設定速度的 send_rc_control() 不同。
- 這裡若是希望移動得更快或更慢可搭配 set_speed(X)，X 範圍為 10 ~ 100cm/s

`rotate_clockwise(self, x)`

Rotate x degree clockwise.

Parameters:

Name	Type	Description	Default
`x`	`int`	1-360	*required*

Source code in `djitellopy/tello.py`

`rotate_counter_clockwise(self, x)`

Rotate x degree counter-clockwise.

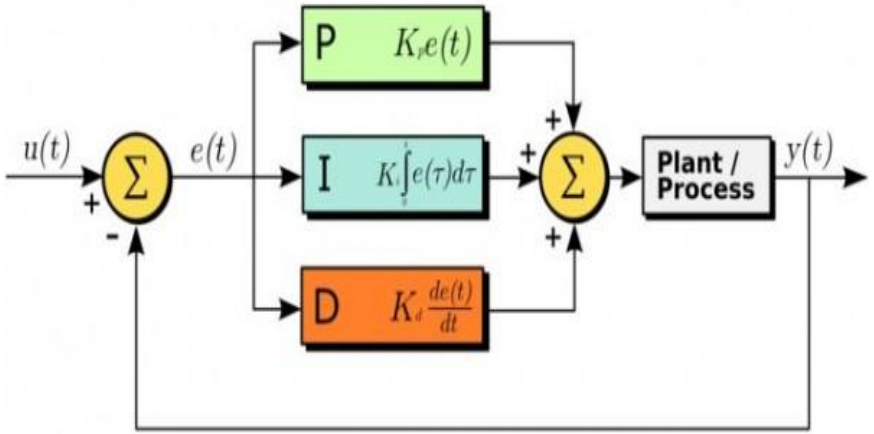
Parameters:

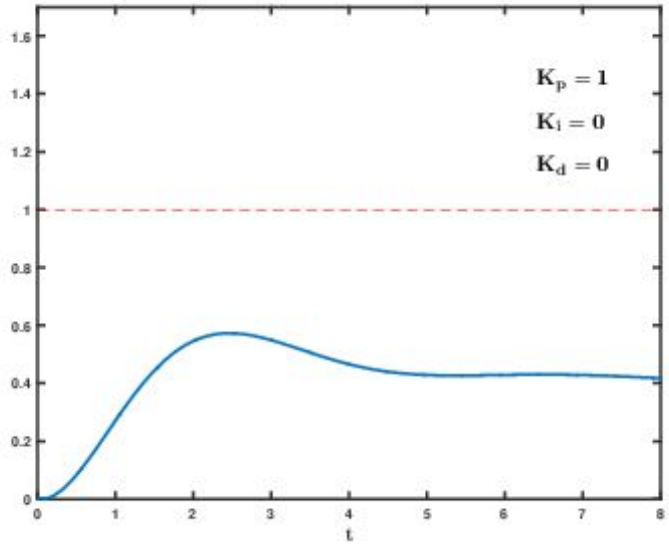
Name	Type	Description	Default
`x`	`int`	1-360	*required*

PID Control

Theory

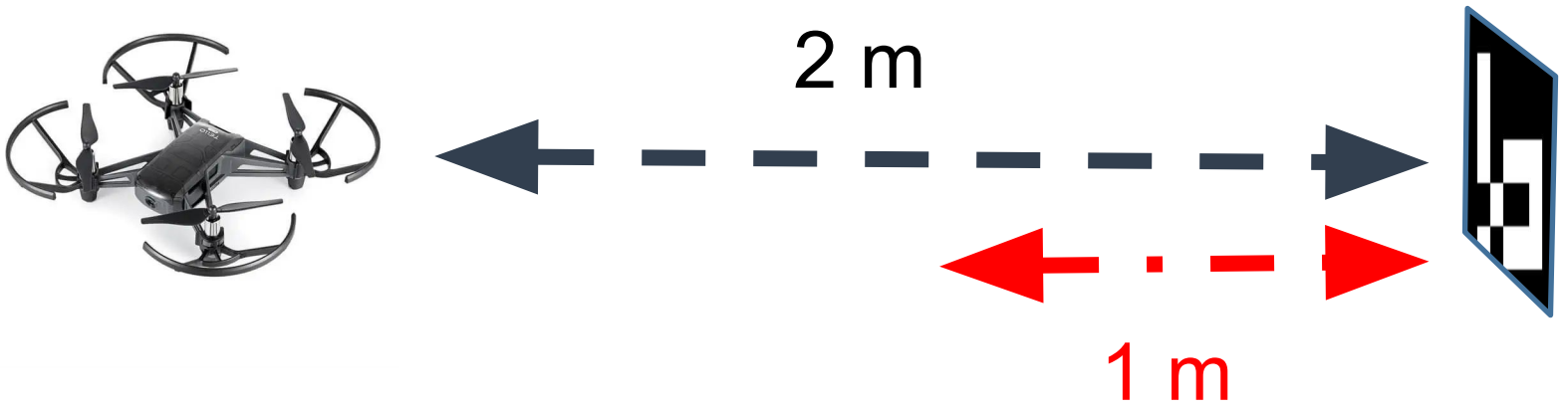
$$\mathrm{u} = \underbrace{K_p e}_{\text{Proportional Term}} + \underbrace{K_i \int_0^t e\,dt}_{\text{Integral Term}} + \underbrace{K_d \frac{d}{dt} e}_{\text{Differential Term}}$$





調整方式	上升時間	超調量	穩態誤差	穩定性
↑ K_p	減少 ↓	增加 ↑	減少 ↓	變差 ↓
↑ K_i	小幅減少 ↘	增加↑	大幅減少↓↓	變差↓
↑ K_d	小幅減少 ↘	減少↓	變動不大→	變好 ↑

Step 1



欲修正的誤差 (error) : 2 m – 1 m = 1 m

Step 2



利用 PID 去 smooth 原本的誤差

1 m ⟶ 0.4 m

將誤差轉換成速度給無人機

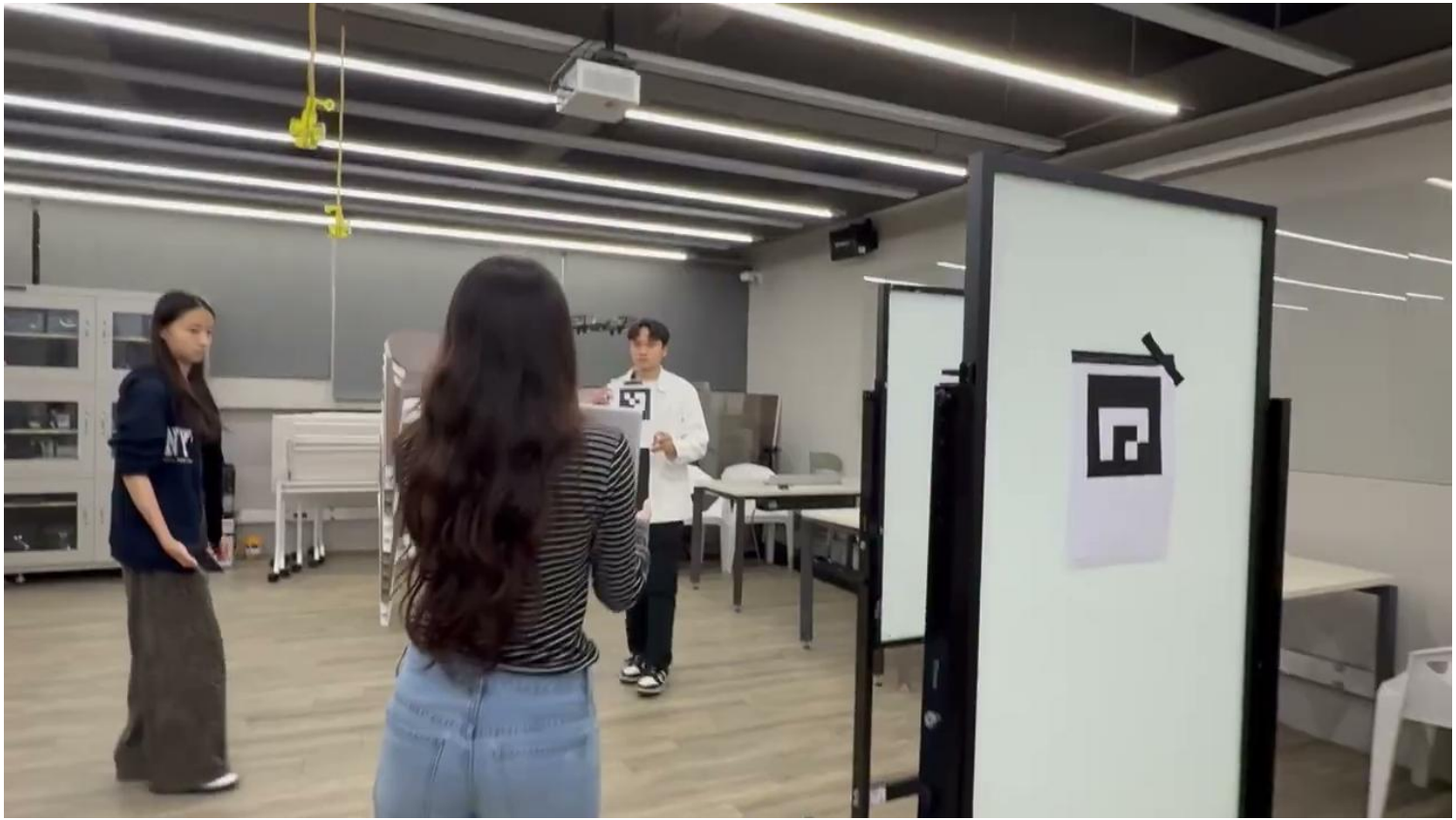
Step 3

根據無人機飛行的狀況調整 PID



1. 先把 I, D 設為 0
2. P : 無人機會停在你設定的距離附近
3. I : 無人機會在設定的距離附近抖動
4. D : 停止抖動

範例



範例



pyimagesearch

- 將 pyimagesearch 資料夾複製到與 lab6.py 同目錄下



名稱	修改日期	類型	大小
__pycache__	2022/3/29 下午 02:13	檔案資料夾	
__init__.py	2020/6/16 下午 12:01	PY 檔案	0 KB
objcenter.py	2020/6/16 下午 12:01	PY 檔案	2 KB
pid.py	2020/6/16 下午 12:01	PY 檔案	2 KB

```
import cv2
import numpy as np
#import tello
import time
import math
from djitellopy import Tello
from pyimagesearch.pid import PID
```

pyimagesearch

- 在main中宣告會用到的pid

```
def main():
    # Tello
    drone = Tello()
    drone.connect()
    #time.sleep(10)

    global is_flying
    # Get the parameters of camera calibration
    fs = cv2.FileStorage("calibrateCamera.xml", cv2.FILE_STORAGE_READ)
    intrinsic = fs.getNode("intrinsic").mat()
    distortion = fs.getNode('distortion').mat()

    z_pid = PID(kP=0.7, kI=0.0001, kD=0.1)
    y_pid = PID(kP=0.7, kI=0.0001, kD=0.1)
    yaw_pid = PID(kP=0.7, kI=0.0001, kD=0.1)

    yaw_pid.initialize()
    z_pid.initialize()
    y_pid.initialize()
```

kP, kI, kD的值可自己調整，但別設太大

pyimagesearch

- 使用時, 先算出想要丟給無人機的速度, 算出其pid值之後再丟給無人機

```
z_update = tvec[i,0,2] - 100
print("org_z: " + str(z_update))
z_update = z_pid.update(z_update, sleep=0)
print("pid_z: " + str(z_update))
if z_update > max_speed_threshold:
    z_update = max_speed_threshold
elif z_update < -max_speed_threshold:
    z_update = -max_speed_threshold
drone.send_rc_control(0, int(z_update//2) ,0 ,0 )
```

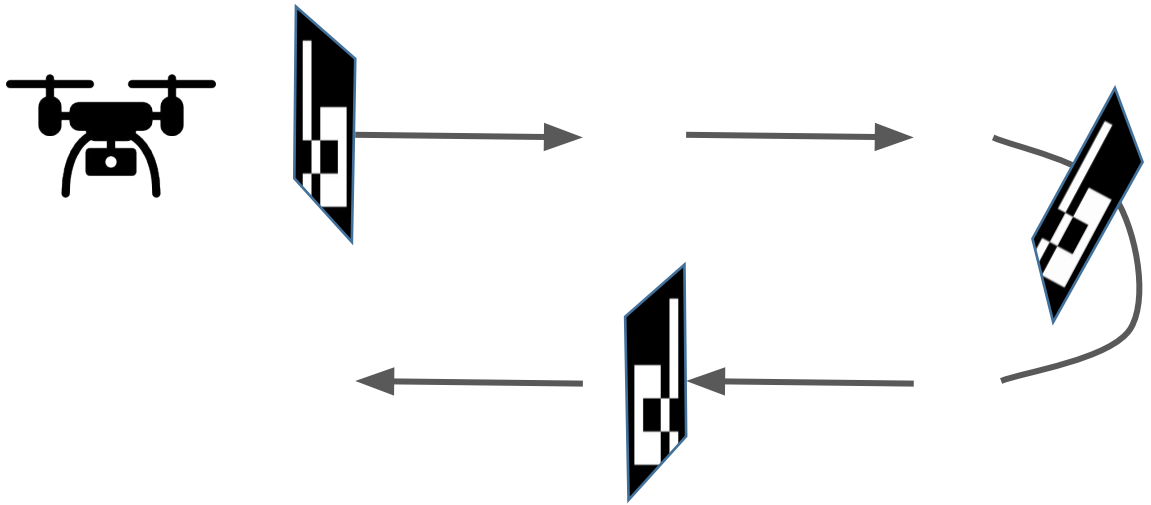
請務必限制最高速度防止失控

Lab6

追蹤 marker 移動

1. 確保鍵盤能控制無人機移動 (20%)

2. 無人機追蹤人手持的 marker 移動，六個方向 + 轉向都要可以正常運作，並且最後測出離maker的距離 (80%)

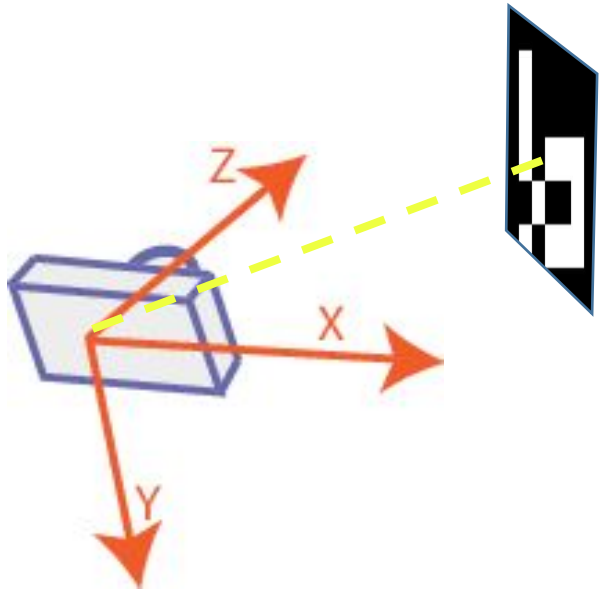


估計無人機與marker的距離

- tvec : 上次lab估距離
- rvec : 旋轉向量
- dst = cv2.Rodrigues(src) # 轉為旋轉矩陣

1. 用 R 乘以 Z 軸 (0, 0, 1) 得到 Z’
2. 將 Z’ 投影到 XZ 平面得到向量 V
3. 求出 Z 與 V 的夾角 (rad) 轉換成 degree

   math.atan2(z, x)



注意事項

撰寫自動飛行的程式碼時, 一定也要有 keyboard control 功能, 且要有**最高優先權**, 確保自動飛行狀況不佳時仍能手動控制。

Lab期間, 前7組在EA101, 後8組去EAB06